

Object Oriented Programming Lab (CL1004)

Date: 06/05/2025

Course Instructor(s)

Mr. Muhammed Monis, Mr. Abullah Shaikh, Ms. Almas
Ayesha Ansari, Ms. Alishba Subhani

Lab Final Exam (B)

Total Time: 120 minutes

Total Marks: 50

Total Questions: 03

Semester: SP-2025

Campus: Karachi

Dept: Computer Science

Comment is your best friend

LLO 03: Demonstrate advanced C++ OOP by implementing inheritance with access modifiers and virtual inheritance; compile-time polymorphism through function and operator overloading; run-time polymorphism via virtual functions, overriding, abstract classes, and pure virtual functions; and utilizing friend functions and classes for controlled flow.

Q1. [15 marks]

A hospital wants to build a staff performance monitoring system. It employs **Doctors**, **Nurses**, and **Resident Doctors** (who are in training and perform both roles). Each role has unique responsibilities and performance metrics. The system must ensure encapsulation of sensitive data while allowing HR to review staff metrics for evaluation.

Requirements:

Base Class – HospitalStaff – 2.5 points

- **Data Members:**
 - staffID (string): Unique staff identifier
 - name (string): Full name of the staff member
- **Function:**
 - displayInfo() – virtual: Shows ID and name

Doctor Class – 2.5 points

- **Additional Members:**
 - patientsTreated (int)
 - diagnosisAccuracy (float; 0–100%)
- **Functionality:**
 - Overrides displayInfo() to include doctor metrics
 - Friend function to adjust both fields (patients must be ≥ 0 , accuracy between 0–100)

Nurse Class – 2.5 points

- **Additional Members:**
 - shiftsPerWeek (int; must be ≥ 1)
 - patientSatisfaction (float; 0–5)
- **Functionality:**
 - Overrides displayInfo() to show nurse metrics
 - Friend function to adjust fields (validation required)

ResidentDoctor Class – 2.5 points

- Inherits from both Doctor and Nurse
- Overrides displayInfo() to include all metrics plus a **combined performance score**:
 - Score = diagnosisAccuracy / 10 + patientSatisfaction
- Must resolve potential attribute duplication (e.g., only one staffID and name)

HRModule Class – 2.5 points

- Can inspect performance metrics of Doctors and Nurses (via friend access)
- Cannot modify metrics
- Prevents public access to sensitive fields

Constraints – 2.5 points

- No direct public access to diagnosisAccuracy or patientSatisfaction
- No duplication of staffID or name in derived classes
- Use clean class hierarchy and ensure role-specific responsibilities are preserved

Sample Input:

```
Doctor doc1("D001", "Dr. Haris", 120, 96.5f);
Nurse nurse1("N002", "Nurse Sarah", 5, 4.7f);
ResidentDoctor rd1("R003", "Dr. Hina", 80, 90.0f, 4, 4.5f);
doc1.displayInfo();
nurse1.displayInfo();
rd1.displayInfo();
adjustDoctor(doc1, 130, 98.0f);
adjustNurse(nurse1, 6, 4.8f);
std::cout << "\nAfter Adjustments:\n";
doc1.displayInfo();
nurse1.displayInfo();
HRModule hr;
hr.inspectDoctor(doc1);
hr.inspectNurse(nurse1);
```

Sample Output:

Doctor Info:

ID: D001, Name: Dr. Haris

Patients Treated: 120

Diagnosis Accuracy: 96.5%

Nurse Info:

ID: N002, Name: Nurse Sarah

Shifts Per Week: 5

Patient Satisfaction: 4.7

Resident Doctor Info:

ID: R003, Name: Dr. Hina

Patients Treated: 80

Diagnosis Accuracy: 90%

Shifts Per Week: 4

Patient Satisfaction: 4.5

Combined Performance Score: 13.5

After Adjustments:

Doctor Info:

ID: D001, Name: Dr. Haris

Patients Treated: 130

Diagnosis Accuracy: 98%

Nurse Info:

ID: N002, Name: Nurse Sarah

Shifts Per Week: 6

Patient Satisfaction: 4.8

HR Module Report:

Doctor Metrics – Treated: 130, Accuracy: 98

Nurse Metrics – Shifts: 6, Satisfaction: 4.8

LLO 03: Demonstrate advanced C++ OOP by implementing inheritance with access modifiers and virtual inheritance; compile-time polymorphism through function and operator overloading; run-time polymorphism via virtual functions, overriding, abstract classes, and pure virtual functions; and utilizing friend functions and classes for controlled flow.

Q2. [15 marks]

You are developing a system to monitor and manage a fleet of **Autonomous Delivery Vehicles**. Each vehicle tracks its model, energy usage, and operational characteristics. Newer vehicle types have added capabilities such as cargo handling and flight control.

Base Requirements for All Vehicles – 3 points

Every vehicle must store:

- model (string): The vehicle model name (e.g., "AutoVan X3")
- batteryUsage (int): Initialized to 10% when created

All vehicles must implement a core function runOperation() that:

- Prints an activity specific to the vehicle type
- Increases battery usage by a fixed amount per operation
- Caps battery usage at 100%
- Implements a statusDisplay() method showing model and battery usage

Specialized Vehicle Types – 5 points

AutoVan Series – 1.5 points:

- Activity: "Delivering ground package"
- Battery increase: +20% per operation

AutoBike Series – 1.5 points:

- Activity: "Delivering fast-food order"
- Battery increase: +30% per operation

AutoDrone Series – 2 points:

- Additional property: altitudeLevel (starts at 50 meters)
- Activity: "Aerial delivery at [altitudeLevel] meters"
- Battery increase: +25% per operation
- Special functionality: Overload ++ to increase altitudeLevel by 10 meters
- Status must display both battery and altitude

System-wide Features – 7 points

- Overload > to compare which vehicle used more battery
- Overload << to print status in the format:
[Model]: [Battery] % (or ...%, Altitude: [value]m for drones)
- Demonstrate polymorphism via base-class pointers or references

Constraints – 2 points

No multiple inheritance, No hardcoded values in global scope, Avoid redundancy; ensure clean inheritance and operator design

Sample Input:

```
Vehicle* fleet[3];
fleet[0] = new AutoVan("AutoVan X3");
fleet[1] = new AutoBike("AutoBike Swift");
fleet[2] = new AutoDrone("AutoDrone SkyEye");
for (int i = 0; i < 2; ++i)
    fleet[i]->runOperation();
++(*fleet[2]); // raise drone altitude
fleet[2]->runOperation();
for (int i = 0; i < 3; ++i)
    std::cout << *fleet[i] << std::endl;
if (*fleet[1] > *fleet[0])
    std::cout << fleet[1]->getModel() << " consumed more battery than " << fleet[0]->getModel();
```

Sample Output:

```
AutoVan X3 is Delivering ground package... AutoVan X3 is Delivering ground package...
AutoBike Swift is Delivering fast-food order... AutoBike Swift is Delivering fast-food order...
AutoDrone SkyEye is Aerial delivery at 60 meters...
AutoVan X3: 50 %
AutoBike Swift: 70 %
AutoDrone SkyEye: 35 %, Altitude: 60m
AutoBike Swift consumed more battery than AutoVan X3
```

LLO 04: Demonstrate proficiency in C++ generic programming and robust error management by implementing function and class templates, handling exceptions, and performing practical file operations and IOstream-based input/output.

Q3 [20 marks]

Ali is building a Smart Fleet Management System for an autonomous transportation company. The system is responsible for managing different types of vehicles, including delivery vans, passenger cars, and autonomous drones. Each vehicle has a unique identifier, a top speed, and an energy level (e.g., fuel or battery percentage). Some types of vehicles also require special operating permissions (e.g., for drones or heavy vehicles), while others can be used without any restrictions.

To ensure the system is scalable and easy to manage, Ali plans to design a solution that works efficiently for different types of vehicles, ensures secure operation through permission validation, and provides mechanisms for saving/loading fleet information to and from files.

Mentioned Below are the requirements

1. Define a general-purpose base class for vehicles (2 points)

Create a base class to represent vehicles with a function to return the vehicle ID, A function to return the maximum speed of the vehicle and a function to return the energy level (e.g., fuel or battery). Ensure that specific vehicle types (like delivery vans, passenger cars, and drones) provide their own implementations of these behaviors.

2. Implement multiple vehicle types (2 points)

Create classes for DeliveryVan (include additional attributes like cargo capacity), PassengerCar (include number of seat and AutonomousDrone (include maximum flight altitude) Each type must provide relevant data and behavior.

3. Add a mechanism for permission validation (2 points)

Implement a separate class that holds a list of permission codes required to operate certain vehicles (e.g., heavy-duty or airspace licenses). This class should have a function to check if a user's list of credentials includes all required permissions.

4. Introduce extended vehicle types (2 points)

Create additional vehicle types such as ElectricDeliveryVan (tracks battery level instead of fuel), HybridPassengerCar (has a mode switch between electric and fuel). These types should override behavior where necessary and add relevant member variables.

5. Develop a manager class to store and operate on a vehicle collection (3 points)

Create a manager class that should Add new vehicle objects to the fleet, Display details of all vehicles, Calculate and return the average energy level across all vehicles. Verifying if a user is allowed to operate a particular vehicle based on their credentials.

6. Integrate file input/output and exception handling (4 points)

Extend your manager class to support saving the current fleet to a text or binary file, Loading a fleet from a file and reconstructing the state, Handling potential issues such as missing files, malformed data, or duplicate vehicle IDs using a custom exception class.

7. Demonstrate system usage (5 points)

In your main function instantiate and add different vehicle types to the manager class.

1. Save the fleet to a file, then load it back.

2. Display all vehicle details.

3. Demonstrate a case where a user is allowed and another case where a user is denied access to operate a vehicle.

4. Catch and handle any exceptions that occur during file loading or eligibility checks.

Sample Input:

Fleet.txt:

Format: VehicleType,ID,Speed,Energy,[ExtraData...]

DeliveryVan,VN101,120,40,Cargo:500

PassengerCar,PC202,180,70,Seats:5

AutonomousDrone,DR303,90,85,Altitude:500

ElectricDeliveryVan,EV404,110,88,Battery:88

HybridPassengerCar,HC505,160,50,Mode:Hybrid

Console Input/output:

Display the content of the file at the start of the program

Sample Output:

//When checking user eligibility

Enter user ID: USER001

Enter user credentials (comma-separated): DRONE_CERT,HEAVY_VEHICLE

Eligibility Check:

- Can operate DR303 (AutonomousDrone): Yes

- Can operate VN101 (DeliveryVan): Yes

- Can operate EV404 (ElectricDeliveryVan): Yes

- Can operate HC505 (HybridPassengerCar): Yes

Enter user ID: USER002

Enter user credentials (comma-separated): NONE

Eligibility Check:

- Can operate DR303 (AutonomousDrone): No - Missing: DRONE_CERT

- Can operate VN101 (DeliveryVan): Yes

- Can operate EV404 (ElectricDeliveryVan): Yes

- Can operate HC505 (HybridPassengerCar): Yes

When Displaying Average Energy Level:

Average Energy Level Across Fleet: 66.6%

On Exception (e.g., malformed file or duplicate ID):

Loading fleet from file: fleet_corrupt.txt

Error: FleetParseException - Duplicate Vehicle ID 'PC202' found.